

프로젝트 최종보고서

학생 팀별 작성용

과제 수행원 현황						
수행 학기	2025년 9월 ~ 2025년 12월					
프로젝트명	바이링_해외구매 간소화 서비스					
팀명	바이링					
	학과	학번	성명	성별	연락처	E-mail
팀장	멀티미디어소프트웨어공학전공	2020112534	강병민	남	010-8715-4894	flareon078@naver.com
팀원	컴퓨터공학전공	2020111780	남윤수	남	010-8277-2023	angusnam1352@gmail.com
	컴퓨터공학전공	2023112399	박지은	여	010-9086-2703	zienyyyy@gmail.com
	컴퓨터공학전공	2023112246	최하영	여	010-7136-3761	hayo021111@naver.com
교과목 / 동아리	교과목명	공개 SW 프로젝트				
	지도교수	송수환 교수 / 컴퓨터·AI 학부				
	동아리					
산업체 멘토	기업명					
	멘토 성함		멘토 직위			

과제 일반 현황					
작품(과제)명	바이링_해외구매 간소화 서비스				
포상여부	상격	기관	행사명	수상일시	부상내역
	※ 포상실적은 해당사항이 있을시 필히 기재 요망. ※ 포상실적을 허위로 기재시 신청인은 포상대상에서 제외됨 ※ 타기관에서 이미 수혜받은 정부포상 과제는 포상대상에서 제외됨				

보고서					
작품명 (프로젝트명)	바이링_해외구매 간소화 서비스				
# Key Words					
1. 프로젝트 개요	1. 개발동기 및 목적 가. 솔루션 제안 : 복잡한 해외 BUY, 링크 하나로 바이링! ■ AI 이미지 무게/부피 계산을 통해 예측 배송비로 '2차 결제' 과정 생략 ■ 상품 정보 크롤링과 자동 견적 계산을 통해 사람이 직접 CS하며 걸리던 평균 10분이 걸리는 절차를 10초로 단축  1. 복잡한 절차 해결 • 링크 하나만 있으면 상품 정보 자동 호출 기존 8단계 => 4단계로 단축 • AI 이미지 기반 무게/부피 예측을 통해 2번의 결제 절차를 1번으로 단축 불투명한 비용 해결 • AI 이미지 기반 무게/부피 예측을 통해 장바구니에서 상세 견적 즉시 확인 가능 ■ 바이링 UI/UX : 견적 확인이 가능한 장바구니 				
	나. 목적 설정 ■ AI 기술을 통해 해외직구의 복잡한 절차와 불투명한 비용 구조를 해결하고, 누구나 쉽고 신뢰 있게 전 세계 상품을 구매할 수 있는 구매대행 플랫폼을 구축한다.				
	다. 목표 설정의 필요성 ■ 시장 환경의 변화 - 24년 기준 국내 해외직구 시장은 7.9조원, 전년 대비 17.8% 상승 - 그러나 여전히 55.9%의 이용자가 불편을 경험하고 있음 ■ 기존 서비스의 한계				

- 직구 플랫폼(테무, 알리 등)은 저가 위주로 품질, 환불 신뢰성이 낮음
- 일반 구매대행 플랫폼(몰테일 등)은 절차가 복잡하며 2차 결제 구조로 비용이 불투명함

■ 기술적 해결의 필요성

- AI 기술의 발전으로 이미지 기반 무게, 부피 예측, 비용 자동선정 가능
- 이를 구매대행 구조에 접목하면 비용, 절차, 시간의 불확실성을 해소 가능

■ 사회적 필요성

- 정보비대칭으로 인해 해외 구매 접근성이 낮은 소비자가 많음
- 바이링은 이를 기술로 해결하여 소비자 권익 보호 및 거래 공정성 강화에 기여

■ 산업적 확장성

- AI 구매대행 기술은 향후 B2B SaaS, 물류 자동화, 글로벌 큐레이션 커머스로 확장 가능함

2. 개발목표 및 범위

가. 세부 목표

■ 기술 목표 : AI 기반 구매 자동화 모델 개발

- 이미지 인식 기반 상품정보 추출 및 무게/부피 예측 가능
- 상품가, 대행비, 배송비, 세금을 포함한 총 비용 실시간 산정 시스템 구축
- AI 모델 구축 후 특허 출원 및 등록 진행

■ 서비스 목표 : 원클릭 결제형 구매대행 서비스 상용화

- 상품 링크나 이미지 한 장으로 주문부터 배송까지 자동화
- 결제, 통관, 환불 절차를 통합한 사용자 중심 UX 설계

■ 시장/비즈니스 목표 : 신뢰도 높은 해외직구 대행 서비스 선도

- 구매대행 사용자 10명 모집 후 피드백 수립
- 인당 객단가 6만원 기준 60만원의 매출과 6만원 (10%) 순수익 확보

■ 사회적 목표 : 해외 소비자 접근성 향상 및 피해 예방

- 절차, 언어, 통관 등 정보 격차 해소
- 해외 구매 피해 (55.9%)를 30% 이하로 낮추는 기술적 대안 제시

나. 개발 및 연구 범위 & 현실적 제한요소 및 타당성

■ AI 견적 예측 모델 개발

- 이미지 분석을 통한 상품의 무게/부피 인식
- 배송비 및 관세 자동 산출 알고리즘 개발
- 주요 성과물 : AI 기반 비용 예측 모델

- 제약 요소 :

- 원가 측면 : 초기 AI 모델 학습 데이터 확보 및 GPU 연산 원가가 높음 => 공개 데이터와 자체 보유 상품 이미지를 병행 학습하며, 학교 인프라를 통해 해결
- 안정성 측면 : AI 예측 결과 오차에 따른 배송비 오류 위험 => 예측 오차 보정 알고리즘으로 실측 데이터 기반 피드백 루프 구축 + 신뢰도 지수로

결과 보고

■ 자동화 결제 및 통관 시스템

- 1회 결제 구조 (상품가 + 예측 배송비 + 세금 등 포함)

- 개인통관고유부호 자동 연동 및 신고 자동화 API 활용 검증 로직 구축

- 주요 성과물 : 결제/통관 자동화 API

■ 사용자 UX/UI 인프라 구축

- 사용자 중심 원클릭 구매대행 UX 설계 : 링크 하나만 입력하면 상품 정보가 불러와지고, 건적 확인이 용이하게 설계

- AWS 기반 백엔드 및 데이터 파이프라인 구축

- 주요 성과물 : 상용화 가능한 MVP

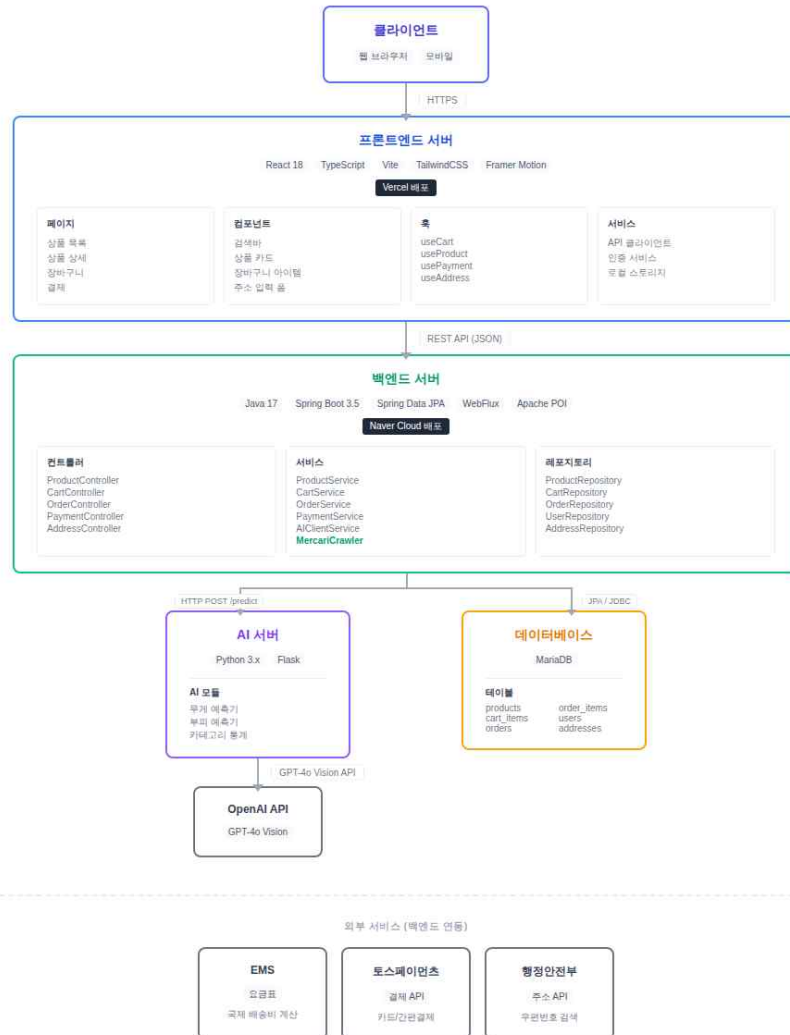
다. 프로젝트 단계별 계획

단계	기간	주요 내용
1단계	기술 기획 및 AI 모델 초기 설계	- 이미지 데이터셋 학습 및 무게 예측 모델 프로토타입 - API 구조 설계 및 기술 검증
2단계	AI 자동건적 결제 시스템 구현	- 결제 · 통관 연동 / UX 프로토타입 구축 - 모델 성능 개선 및 안정성 검증
3단계	서비스 안정화 및 상용화	- 사용자 테스트 진행 - 기술 특허 준비

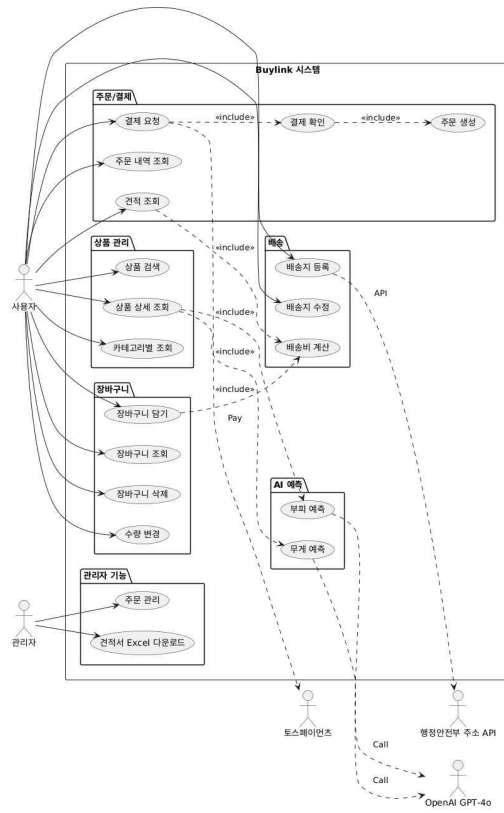
2. 최종 결과물 소개

1. 시스템 아키텍처

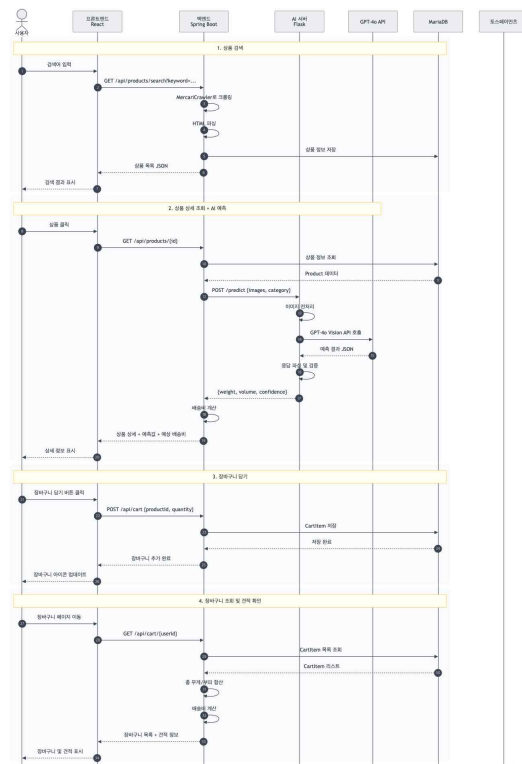
Buylink 시스템 아키텍처

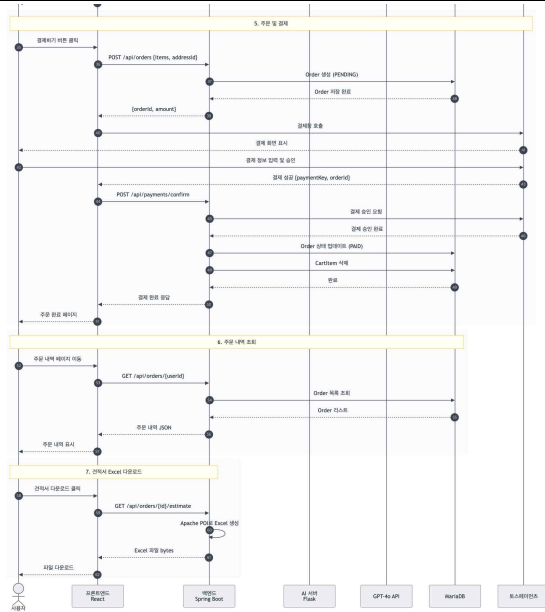


2. 유즈케이스 다이어그램



3. 시퀀스 다이어그램





4. 백엔드

가. API 명세서

- 본 프로젝트의 백엔드 서버는 /api prefix를 사용하는 RESTful API로 구성되며, 상품 정보 크롤링, 장바구니 관리, 결제 처리, 주문 내역 조회까지의 전체 기능을 제공한다. 각 기능은 HTTP 메서드 규칙에 따라 설계되고, 프론트엔드는 해당 API를 기반으로 화면을 구성한다.

구분	주요 기능	Method	URL
상품 정보 불러오기	상품 정보 크롤링	POST	/api/products/fetch
	AI 부피 · 무게 예측	POST	/api/products/predict
장바구니	장바구니 조회	GET	/api/cart
	장바구니 담기	POST	/api/cart
	장바구니 선택 삭제	DELETE	/api/cart?ids=1,3,7
	장바구니 전체 삭제	DELETE	/api/cart?all=true
	장바구니 견적 계산	POST	/api/cart/estimate
결제	도로명 주소 검색	GET	/api/address/search
	배송지 등록	POST	/api/orders/address
	개인통관고유부호 검증	POST	/api/orders/customs-code/verify
	결제 요청	POST	/api/orders/pay
주문내역	주문 생성	POST	/api/orders
	주문 상세 조회	GET	/api/orders/{orderId}?receiver={receiver}&phone={phone}

표 3 주요 API의 구분, 기능, HTTP 메서드, URL

나. 공통 기반 정리

- 본 프로젝트의 백엔드 서버는 공통 패키지 구조와 응답 포맷, 예외 처리, CORS 설정, 외부 서버 연동 구조를 기반으로 일관된 동작을 유지하도록 구성하였다.
- 모든 API는 공통 응답 포맷을 사용하며, 예외 상황은 전역 예외 처리기를 통해 표준화된 형태로 반환된다. 또한 프론트엔드 및 외부 크롤링·AI 서버와의

통신을 고려하여 CORS 및 WebClient 기반 연동 구조를 적용하였다.

- 각 공통 구성 요소의 구체적인 구현 방식과 처리 흐름은 이후 백엔드 구현 과정에서 상세히 설명한다.

다. 상품 정보 불러오기

- 상품 정보 불러오기 기능은 사용자가 입력한 상품 URL을 바탕으로 Mercari 상품 정보를 수집하고, AI 예측값을 생성하여 프론트엔드에 전달하는 구조로 이루어진다.
- 상품 정보 조회- 사용자가 상품 URL을 제출하면 /api/products/fetch에서 Python 크롤러 서버로 URL이 전달되며, 크롤러는 제목, 가격, 이미지, 카테고리, 배송비 포함 여부, 품질 여부 등을 수집한다. 수집된 데이터는 ProductInfoDto로 변환되어 반환된다.
- AI 기반 무게·부피 예측- /api/products/predict 요청을 통해 상품명·카테고리·이미지 데이터를 AI 서버로 전달하여 예상 무게·부피 값을 생성한다. 이 값은 장바구니 저장 및 견적 계산 단계에서 활용된다.
- 페이지 간 데이터 연동- 조회된 상품 정보는 CartAddRequest에 그대로 반영되며, 장바구니 API로 전달되어 이후 단계에서도 동일한 데이터 구조를 유지한다.

라. 장바구니

- 장바구니 기능은 사용자가 담은 상품 정보를 proxy_sid 기준으로 저장하며, 조회·삭제·견적 계산을 통해 결제 전에 필요한 정보를 제공한다.
- 상품 추가- 사용자가 장바구니 담기를 요청하면 서버는 /api/cart로 전달된 CartAddRequest를 저장한다. 상품명, 가격, 이미지, 배송비 여부뿐 아니라 AI 예측값도 함께 기록된다. 동일 URL의 중복 저장은 방지된다.
- 장바구니 조회- /api/cart GET 요청을 통해 저장된 모든 상품과 총금액이 반환된다. 각 상품 데이터는 이후 견적 계산 및 주문 생성 단계에서 그대로 사용된다.
- 상품 삭제- 삭제는 /api/cart?ids= 형태로 처리되며, 결과는 DeleteResponse로 반환된다.

마. 결제

- 결제 기능은 배송지 정보 저장, 통관번호 검증, 주문 생성, 결제 승인 과정을 포함하며, 장바구니 견적 금액과 실제 결제 금액의 일치 여부를 검증한다.
- 배송지 및 통관번호 처리- 배송지는 /api/orders/address로 저장되며, 통관번호는 /api/orders/customs-code/verify에서 형식 검증 후 Boolean 값으로 반환된다.
- 주문 생성- /api/orders 요청 시 서버는 장바구니 견적값과 사용자가 제출한 결제 금액을 비교하고, 금액이 일치하면 주문을 생성하여 주문번호를 부여한다.
- 결제 승인 처리- 토스 결제 승인 요청은 /api/orders/pay를 통해 처리되며, 승인 성공 시 주문 상태가 결제 완료로 확정된다. 실패 시 표준화된 오류 응답이 반환된다.

바. 주문내역

- 주문내역 페이지는 사용자가 결제를 완료한 주문의 상세 정보를 조회할 수 있도록 구성된 기능으로, 주문 생성 당시의 데이터를 스냅샷 형태로 저장하여 이후 변경에 영향을 받지 않도록 한다. 이를 통해 사용자는 결제 완료 후에도 동일한 내용으로 확인할 수 있다.
- 주문 조회- 사용자가 주문번호를 요청하면 서버는 `/api/orders/{orderId}`를 통해 해당 주문 정보를 조회한다. 반환되는 데이터에는 수취인, 연락처, 주소, 배송 요청 사항, 상품 금액, 배송비, 수수료, 최종 결제 금액 등 결제 시점의 모든 항목이 포함된다.
- 결제/배송 관련 정보 유지- 주문 정보는 생성 시점에 저장된 값이 그대로 유지되므로, 장바구니 상태나 상품 정보가 이후 변경되더라도 기존 주문내역에는 영향을 주지 않는다. 이를 통해 사용자는 언제든지 정확한 결제 기록을 확인할 수 있다.

3. 프론트엔드

가. 전체 구조 및 상태 관리

- 프론트엔드는 단일 페이지 애플리케이션(SPA)으로 구성해 메인-구매대행 요청-장바구니-결제-주문 완료-주문 내역 조회까지 새로고침 없이 이어지도록 했다. 모든 HTTP 요청에 공통 규칙과 세션 쿠키(proxy_sid)를 적용해, 로그인 없이도 한 브라우저 안에서는 장바구니와 주문 흐름이 유지되는 “회원가입 없는 해외 구매대행” 경험을 제공한다.

나. 레이아웃 및 내비게이션 구조

- 공통 레이아웃 컴포넌트에서 헤더·배경·본문 최대 너비를 일괄 관리해 페이지마다 디자인이 달라 보이지 않도록 했고, 상단 고정(sticky) 헤더와 로고 클릭 시 메인(/) 이동으로 언제든지 초기 화면으로 돌아올 수 있게 했다. “주문내역 확인하기” 버튼은 홈에서만 노출되어 `/orders`로 이동하며, 장바구니-결제-주문 완료-주문 조회는 동일 레이아웃 안에서 내용만 바뀌는 구조로 설계했다.

다. 구매대행 요청 페이지(RequestPage)

- 사용자는 메루카리 등 해외 쇼핑몰 상품 URL을 입력해 상품명·가격·이미지·카테고리·품질 여부·배송비 포함 여부를 카드 형태로 조회할 수 있다. 동일 URL은 첫 번째만 구매 가능, 이후 상품은 품질 처리해 중복 장바구니 추가를 막고, “장바구니에 담고 견적 확인하기” 버튼으로 선택 상품을 장바구니로 보내 견적 단계로 자연스럽게 이동하도록 했다.

라. 장바구니 페이지(CartPage) & 견적 컴포넌트(CartQuotation)

- 장바구니 페이지에서는 세션 기준으로 저장된 상품의 이름·가격·이미지·수량·AI 예측 무게·부피를 한 번에 확인하고, 전체/부분 선택, 선택 상품 삭제, 개별 삭제 기능으로 장바구니를 관리할 수 있다. 오른쪽 CartQuotation 패널에서는 선택된 상품과 추가 포장·보험 옵션을 반영한 상품 총액, 수수료, 해외·국내 배송비, 포장·보험 비용, 최종 결제 금액을 보여주고, 예측 무게·부피를 $g \cdot cm^3$ 로 변

환해 “AI가 어떤 기준으로 배송비를 계산하는지”를 직관적으로 제시하며, “총 결제 예상 금액 [금액]원” 버튼으로 /checkout으로 이동한다.

마. 결제 페이지(CheckoutPage) 및 모달 컴포넌트

- 결제 페이지는 배송지 정보, 개인통관고유부호, 최종 결제 금액 확인, 약관 동의 한 화면에서 처리하고, 진입 시 장바구니와 견적을 다시 조회해 화면에 보이는 금액과 실제 결제 금액을 맞춘다. 배송지는 AddressModal에서 주소 검색과 입력 검증 후 저장하고, 개인통관고유부호는 CustomsCodeModal에서 형식 검증·서버 검증 후 주문 생성에만 사용하며, “결제하기” 클릭 시 필수 정보와 금액 유효성을 모두 통과했을 때에만 TossPayments 결제 모듈을 호출해 카드 결제를 진행한다.

바. 결제 성공 처리 페이지(PaymentsSuccessPage)

- 결제 성공 콜백에서는 paymentKey, orderId, amount를 받아 먼저 백엔드에 결제 검증을 요청해 실제 승인 여부를 확인한다. 승인된 경우에만 이전에 저장해 둔 배송지·개인통관고유부호와 함께 주문 생성 API를 호출해 최종 주문번호를 발급받고, “결제 결과 처리 중입니다” 화면에서 끊임 없이 주문 완료 페이지로 이동하도록 구성했다.

사. 주문 완료 페이지(OrderCompletePage)

- 주문 완료 페이지는 결제 후 발급된 주문번호로 주문 정보를 다시 조회해 수령인, 배송지, 요청사항, 결제 수단, 상품 목록, 비용 내역(상품 금액·수수료·배송비·포장/보험·최종 결제 금액)을 블록 단위로 보여준다. 상단에는 주문번호 복사 기능을, 하단에는 “추가로 구매대행 요청”과 “홈으로 가기” 버튼을 두어 주문 직후 바로 다음 행동을 선택할 수 있게 했다.

아. 주문 내역 조회 페이지(OrderHistoryPage)

- 주문 내역 조회 페이지에서는 이름·전화번호·주문번호만으로 기존 주문을 다시 확인할 수 있으며, 형식이 잘못된 경우 첫 번째 오류만 안내해 사용자가 어디를 수정해야 하는지 바로 알 수 있도록 했다. 조회에 성공하면 주문 완료 페이지와 동일한 구조로 배송지·상품·결제 정보를 보여줘 “주문 직후 화면”과 “나중에 조회하는 화면”의 경험을 통일하고, 실패 시에는 안내 문구와 함께 메인 또는 구매대행 요청 페이지로 이동할 수 있는 버튼을 제공한다.

4. ai

가. 1.1 프로젝트 개요 및 목표

- 본 프로젝트는 해외구매대행 서비스 '바이링(Buylink)'의 핵심 기능인 배송비 자동 산정을 위해, 제품 이미지만으로 무게와 부피를 예측하는 AI 시스템을 개발하는 것을 목표로 함
- 핵심 목표: 무게 예측 오차 500g 이내
- 배송비 구간 정확도 최대화

나. 1.2 데이터 수집 계획

- AI 모델 학습 및 평가를 위해 두 가지 데이터셋을 수집할 계획이다.
- (가) AIHub 물류공간 예측 데이터셋
 - AI Hub에서 제공하는 물류공간 예측 데이터셋을 활용하여 Image2Mass 모델을 학습

- (나) 메루카리(Mercari) 크롤링 데이터셋
 - 일본 최대 중고거래 플랫폼 메루카리에서 실제 판매 상품 데이터를 수집할 계획
 - 수집 목적: 실제 서비스 환경과 유사한 테스트 데이터 확보 및 카테고리별 무게/부피 통계 분석
- (다) 카테고리별 통계 분석 계획
 - 수집된 메루카리 데이터를 카테고리별로 분류하여 평균 무게, 평균 부피, 표준편차 등의 통계를 산출
 - MAST3R 스케일 보정: 3D 복원 시 절대 스케일을 알 수 없으므로, 카테고리별 평균 부피를 참조값으로 사용
 - GPT 프롬프트 참조: GPT-4o에게 카테고리별 평균값을 제공하여 예측 정확도 향상

다. Stage 1-2 시스템 설계

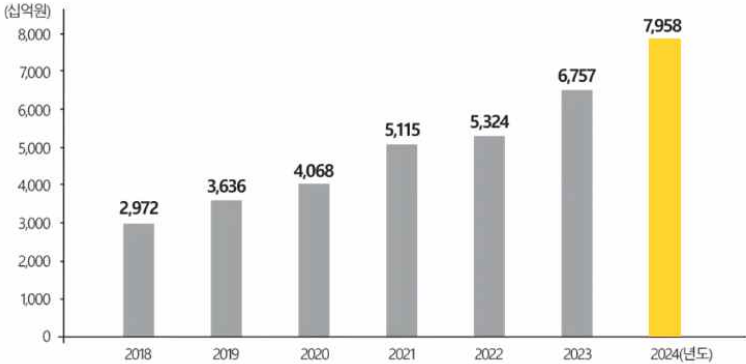
- 판매자가 무게/부피 명시한 경우 사용하는 규칙 기반 텍스트 추출 시스템 (Stage 1) 개발
- Stage 1가 실패 경우 사용하는 제품명과 사진을 이용해 웹서치를 이용한 공식 제품 무게/부피 검색 기능 (Stage 2) 개발
- OCR 양상블 및 이미지 검색을 통해 AI 모델 없이도 높은 정확도로 무게·부피를 추정할 수 있는 전처리 시스템을 구축

라. Stage 3 Image2Mass 모델

- 모델 설계
 - 3D 복원 기반 부피 추정 후 무게를 예측하는 방식 설계
 - MAST3R: Multi-View 이미지에서 3D 포인트 클라우드 복원
 - Image2Mass: Xception 백본 기반 밀도/부피 예측 무게 계산
 - 버전1: 기본(MAST3R + Image2Mass)
 - 버전2: 카테고리 예측 모델 추가
 - 버전3: 카테고리 정답값 사용
- 모델 학습
 - AI Hub 물류공간 데이터셋을 활용하여 Image2Mass 모델을 학습
 - 학습 데이터: 122,378건(8:1:1 분할)
 - MAST3R 스케일 보정을 위한 카테고리별 평균 부피값 참조 전략
- 모델 평가
 - 메루카리 1,000 샘플을 대상으로 3가지 버전의 성능을 평가
 - 평가 지표: 성공률, MAE(g), 500g+ 오차율
 - 버전별 비교 분석

마. Stage 3 GPT 모델

- 모델 설계
 - GPT-4o의 비전 기능과 사전 학습된 제품 지식을 활용하는 방식 설계
 - RAG, Few-Shot 등 여러 방법으로 설계
 - GPT-4o Vision API 활용
 - 카테고리 통계 정보 제공 전략

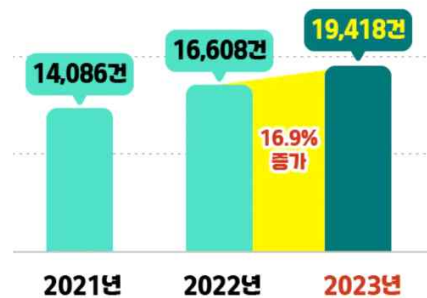
	- 프롬프트 엔지니어링을 통한 성능 최적화 ■ 모델 학습 - GPT-4o는 사전 학습된 모델이므로 별도 학습 없이 프롬프트 최적화로 성능을 개선 - 메루카리 카테고리별 평균/표준편차 통계 수집 - 다양한 프롬프트 버전 실험(Baseline, v2~v5, Few-Shot) ■ 모델 평가 - 메루카리 1,000샘플을 대상으로 6가지 버전의 성능 평가 - 평가 지표: 성공률, MAE(g), 500g+ 오차율 - 버전별 비교 분석 바. 두 모델 성능 비교 ■ Image2Mass 기반 방식과 GPT-4o Vision 방식의 성능을 종합 비교하여 최적 모델을 선정 - 동일 테스트셋(메루카리 1,000샘플)으로 공정 비교 - 실용성, 정확도, 비용 측면 종합 평가
3. 프로젝트 추진 내용	5. 개발 배경 및 요구사항 분석 가. 해외 구매 시장의 성장 ■ 2019년 3조 6000억(5,625만 건), 2020년 4조 1000억(6,400만 건), 2024년 7조 9천억(1억 2,300만 건), 2025년 10조 예상 국내 해외직구 시장 규모  자료 : 통계청, 하나증권 ■ 2018년 대비 2024년 총 6년간 시장 규모 약 2.7배 성장

나. 해외 구매를 하는 이유

- 해외 구매 경험자 1,000명 대상 설문조사 진행 (한국소비자원, 2018)
- 국내에서 판매하는 제품보다 저렴해서 (국내 대비 27.7% 저렴)
- 국내에서 판매하지 않기 때문에

다. 해외 구매의 불편함

- 해외 구매 경험 소비자 1,000명 중 55.9%가 불만 또는 피해를 경험했다고 응답
- 최근 3년간 국제 거래 소비자 불만 상담 접수가 꾸준히 증가(16.9% / 소비자 24)



라. 소비자 불만 이유

- 1. 절차의 복잡함
 - 기존 구매대행 절차 (8단계)
 - 상품 서칭 >> 주문서 작성 >> 견적서 전달 >> 비용 결제 >> 배대지 수령 및 무게 측정 >> 2차 결제 >> 통관 >> 수령
- 2. 불투명한 비용
 - 2차 결제 (배송비, 배송대행비, 합배송비, 재포장비 등) 비용을 알 수 없음
 - 상품이 배송대행지에 도착해서 무게/부피 측정을 한 후 결제하는 구조
- 기타 불만 (언어 장벽/ 반품&하자 등 사후관리)

2) 선행기술 및 사례 분석 :

- 이 프로젝트를 진행하기 위해 필요한 관련 산업, 기술 등의 동향 등은 어떠한가를 기술하는 부분임

6. 백엔드

가. 개발 환경 및 구조

- 개발 언어 및 프레임워크: Java 17 / Spring Boot 3.5.6
- 데이터베이스: H2(local) → MariaDB(dev, prod)
- API 통신: REST 기반 HTTP/JSON
- 주요 라이브러리:
 - Spring Web / Spring WebFlux(WebClient)
 - Spring Data JPA
 - Jackson Databind(JSON)

- Lombok
- 외부 연동 모듈:
 - Python 크롤러 서버(Flask + Selenium)
 - AI 예측 서버(AI 무게·부피 산출)
- 예외 처리: @RestControllerAdvice 기반 전역 예외 핸들러 적용
- 인증 및 식별: 비로그인 세션(proxy_sid) 기반 사용자 구분
- 결제 연동: Toss Payments API(테스트 모드)

나. 공통 기반 정리

- 공통 응답 포맷 구성
 - 서버 전체 API는 ApiResponse 구조(success, data, error)를 사용하여 응답 형식을 통일하였다. 이를 통해 프론트엔드에서 모든 API 결과를 일관된 규칙으로 처리할 수 있도록 하였다.
- 전역 예외 처리
 - @RestControllerAdvice 기반 예외 핸들러를 구성하여 유효성 검증 오류, 잘못된 요청, 서버 내부 오류 등을 JSON 형태로 표준화해 반환하도록 하였다. 기능별 컨트롤러에서 중복되는 예외 처리 코드를 제거하고 유지보수성을 높였다.
- CORS 및 Web 설정
 - 프론트엔드-백엔드 간 원활한 통신을 위해 /api/** 경로에 대한 CORS 허용 범위를 설정하였다. 개발 환경(localhost)과 실제 서비스 환경의 요청 모두가 정상적으로 처리될 수 있도록 도메인을 지정하고, 쿠키(proxy_sid) 전달을 위해 allowCredentials 설정을 적용하였다.
- 외부 서버 연동(WebClient) 구성
 - 상품 정보를 확보하기 위한 Python 크롤러 서버와 AI 예측 서버와의 통신을 위해 WebClient를 별도로 구성하였다.
 - crawlerWebClient: /crawl로 상품 URL을 전달해 크롤링 결과를 확
 - aiWebClient: /api/products/predict 요청을 통해 AI 예측값을 전달받음
- 개발 환경 분리(application.yml)
 - 개발 환경(H2), 운영 환경(MariaDB)을 구분할 수 있도록 데이터베이스 설정과 외부 API 키 값을 분리하였다. 또한 크롤러·AI 서버의 base-url을 환경 변수로 관리하여 배포 시 유연하게 변경할 수 있도록 구성하였다.
- 로그 및 디버깅 지원
 - 요청 흐름 분석과 디버깅 편의성을 위해 proxy_sid 발급 과정에서 주요 정보를 로그로 기록하도록 하였으며, 외부 연동 실패 시 오류 원인을 파악할 수 있도록 예외 메시지를 표준화하였다.

다. 상품 정보 불러오기

- 상품 정보 불러오기 기능은 사용자가 입력한 상품 URL을 기반으로 외부 크롤링 서버와 AI 예측 서버를 연동하여 상품 정보를 수집하고, 예측된 무게·부피 값을 함께 생성하는 구조로 구현하였다. 해당 기능은 상품 조회 이후 장바구니 및 결제 단계까지 이어지는 전체 흐름의 시작점에 해당한다.

■ 상품 정보 조회

- 사용자가 상품 URL을 제출하면 /api/products/fetch 엔드포인트를 통해 외부 크롤링 서버로 요청이 전달되며, 크롤링 서버는 상품명, 가격, 이미지, 카테고리, 배송비 포함 여부, 품질 여부 등의 기본 상품 정보를 수집하여 반환한다.

- 백엔드는 크롤링 결과를 내부 데이터 구조로 변환하여 프론트엔드에 전달하며, 다양한 응답 형태에 대응할 수 있도록 안정적인 데이터 처리 구조를 적용하였다.

■ AI 기반 무게·부피 예측

- 상품 기본 정보 수집 이후 /api/products/predict 요청을 통해 AI 서버에 상품 관련 데이터를 전달하고, 예상 무게·부피 값을 생성한다.

- 생성된 예측값은 장바구니 저장 및 이후 견적 계산 단계에서 활용되도록 설계되었다.

■ 페이지 간 데이터 연동

- 조회된 상품 정보와 AI 예측값은 장바구니 추가 요청 시 그대로 전달되어 저장되며, 이후 견적 계산 및 주문 생성 단계에서도 동일한 데이터 구조가 유지된다.

- 이를 통해 상품 조회부터 결제까지의 전 과정에서 데이터 불일치가 발생하지 않도록 구성하였다.

■ 예외 처리 및 안정성

- 크롤링 실패, 유효하지 않은 URL, 예측 서버 응답 오류 등의 상황에 대해서는 전역 예외 처리기를 통해 표준화된 오류 응답을 반환하도록 하여 시스템 안정성을 확보하였다.

라. 장바구니

■ 장바구니 기능은 사용자가 선택한 상품 정보를 proxy_sid 기준으로 저장하고, 조회·삭제·견적 계산을 통해 결제 전 필요한 정보를 제공하는 역할을 한다.

■ 상품 추가

- 사용자가 장바구니 담기를 요청하면 서버는 상품 기본 정보와 AI 예측값을 함께 저장하여 이후 견적 계산 및 주문 생성 단계에서 동일한 데이터를 활용할 수 있도록 한다.

- 동일한 상품 URL이 이미 장바구니에 존재하는 경우 중복 저장을 방지하여 데이터 일관성을 유지한다.

■ 장바구니 조회

- 장바구니 조회 요청 시 사용자의 proxy_sid를 기준으로 저장된 상품 목록과 총 상품 금액을 반환한다.

- 반환된 장바구니 데이터는 견적 계산 및 결제 단계에서 그대로 사용된다.

■ 상품 삭제

- 상품 삭제 기능은 선택 삭제 및 전체 삭제를 모두 지원하며, proxy_sid 기반 검증을 통해 다른 사용자의 데이터가 삭제되지 않도록 보호 장치를 적용하였다.

■ 예외 처리 및 안정성

- 존재하지 않는 상품 삭제 요청이나 잘못된 요청 값에 대해서는 전역 예외 처리기를 통해 표준화된 오류 응답을 반환하도록 구성하였다.

■ 견적 계산 수행 방법

- 견적 계산은 장바구니에 저장된 상품 정보와 AI 예측값을 기반으로 실제 결제에 필요한 배송비 및 수수료를 산출하는 단계이다.
- 무게 및 부피 기반 계산
각 상품의 실무게와 부피무게를 계산한 뒤, 두 값 중 더 큰 값을 청구무게로 적용한다.
- EMS 국제배송비 계산
청구무게를 기준으로 EMS 요율표에 따라 국제배송비를 산정하며, 해당 요율은 실제 물류 기준을 반영하여 적용하였다.
- 수수료 및 최종 금액 산출
서비스 수수료, 결제 수수료, 포장비 등을 합산하여 최종 결제 금액을 계산한다.
계산된 견적 결과는 이후 주문 생성 단계에서 결제 금액 검증 기준으로 활용된다.
- 데이터 안정성
견적 계산 결과는 별도로 저장되어 결제 요청 시 변조 여부를 검증하는 기준값으로 사용된다.

마. 결제

- 결제 기능은 배송지 정보 저장, 통관번호 검증, 주문 생성, 결제 승인 과정을 포함하며, 결제 금액의 정확성과 데이터 무결성을 중점적으로 고려하여 설계되었다.
- 배송지 및 통관번호 처리
 - 사용자가 입력한 배송지 정보는 유효성 검증 후 저장되며, 통관번호는 별도 검증 절차를 통해 유효 여부만 반환된다.
- 주문 생성
 - 주문 생성 시 서버는 장바구니 견적 금액과 사용자가 요청한 결제 금액을 비교하여 일치하는 경우에만 주문을 생성한다.
 - 생성된 주문 정보는 이후 변경되지 않는 스냅샷 데이터로 저장된다.
- 결제 승인 처리
 - 토스 결제 API와 연동하여 결제 승인을 처리하며, 승인 성공 시 주문 상태를 결제 완료로 확정한다.
 - 승인 실패 시에는 표준화된 오류 응답을 반환하여 프론트엔드에서 적절한 처리가 가능하도록 하였다.
- 결제 금액 검증
 - 주문 생성 단계와 결제 승인 단계에서 각각 금액 검증을 수행하여 결제 금액 변조를 방지하였다. 이를 통해 결제 과정 전반에서 금액 무결성을 확보하도록 설계하였다.
- 예외 처리 및 안정성
 - 존재하지 않는 addressId, 유효하지 않은 통관번호, 금액 불일치, 토스 승인

실패 등 주요 예외 상황에 대한 오류 메시지는 전역 예외 처리기에서 표준화된 `ApiResponseResponse` 형태로 반환된다.

- 주문 생성과 결제 승인 단계에서는 트랜잭션을 적용하여 “주문은 생성되었으나 결제만 실패하는” 불완전 상태가 남지 않도록 구성하였다. 결제 승인 실패 시 주문 상태는 결제 대기에서 벗어나지 않게 유지하며, 사용자가 재시도할 수 있게 한다.

바. 주문 내역

- 주문 내역 기능은 사용자가 결제를 완료한 주문의 상세 정보를 조회할 수 있도록 구성되었다.
- 주문 조회
 - 사용자가 주문번호를 요청하면 서버는 해당 주문의 정보를 조회하여 반환한다.
 - 반환되는 주문 정보는 주문 생성 시점의 배송지, 비용, 무게, 결제 정보가 모두 포함된 스냅샷 데이터이다.
- 데이터 스냅샷 유지
 - 주문 생성 이후 장바구니 상태나 상품 정보가 변경되더라도 기존 주문 내역에는 영향을 미치지 않도록 설계하였다.
 - 이를 통해 사용자는 결제 완료 후에도 항상 동일한 결제 기록을 확인할 수 있다. 따라서 결제 이후에도 주문 데이터의 일관성과 신뢰성을 보장할 수 있도록 하였다.
- 예외 처리 및 안정성
 - 존재하지 않는 주문번호 요청이나 접근 권한이 없는 경우에는 전역 예외 처리기를 통해 표준 오류 응답을 반환하도록 구성하였다.

7. 프론트엔드

가. 기술 스택 및 개발 환경

- 언어: TypeScript
- 라이브러리: React 18
- 번들러/빌더: Vite
- 라우팅: React Router
- 스타일: TailwindCSS
- 애니메이션: motion/react
- HTTP 통신: 브라우저 fetch API
- 결제 모듈: TossPayments JavaScript SDK
- 배포: Vercel

나. 공통 레이아웃 및 라우팅 구조

- 최상위 레이아웃 컴포넌트인 `MainLayout.tsx`에서 공통 헤더와 `<Outlet />`을 정의하였다.
- 헤더 구성
 - 서비스 로고(텍스트) 및 홈 이동 영역: 로고 클릭 시 /(메인 페이지)로 이동한다.

- “주문내역 확인하기”버튼: 홈 화면에서만 노출되며 클릭시 /orders로 이동한다.

■ 라우팅 구성

- / : MainPage - 랜딩/서비스 소개
- /request : RequestPage - 구매대행 요청(상품 링크 입력)
- /cart : CartPage - 장바구니 및 견적 확인
- /checkout : CheckoutPage - 배송지·통관 정보 입력 및 최종 금액 확인
- /payments/success : PaymentsSuccessPage - 결제 성공 콜백 처리
- /order-complete : OrderCompletePage - 결제 완료 후 주문 상세 표시
- /orders : OrderHistoryPage - 과거 주문 내역 조회

다. API 연동 방식 및 환경 분리

■ 프론트엔드는 모든 HTTP 요청을 /api/... 형태의 상대 경로로 전송하고, 실제 호스트/도메인 관리는 환경 변수와 Vercel 리버스 프록시에서 처리하도록 설계하였다.

■ 로컬 개발 환경 (import.meta.env.DEV === true)

- 프로젝트 초반 로컬 개발 단계에서는 별도의 백엔드 서버 없이 Vite 개발 서버만 띄운 상태에서, 더미 데이터/임시 JSON을 사용해 화면 구조와 UI를 먼저 구현하였다.

■ 배포 환경 (Vercel)

- API_BASE_URL는 빈 문자열로 두고 /api/... 요청은 Vercel의 리라이트 설정을 통해 동일 도메인의 백엔드(Serverless Function 또는 프록시)로 전달되도록 구성하였다.

■ 모든 요청은 credentials: "include" 옵션을 사용해 세션 쿠키(proxy_sid)를 포함하도록 하였고, 이를 통해 로그인 없이도 사용자별 장바구니를 구분할 수 있도록 했다. 이처럼 공통 Base URL과 buildApiUrl() 헬퍼 함수로 구조를 통일하여, 각 컴포넌트는 경로(/api/cart, /api/orders/...)만 신경 쓰면 되고, GitHub 커밋 → Vercel 자동 배포만으로 프론트·백엔드가 함께 업데이트되는 배포 흐름을 구축하였다.

라. 구매대행 요청 페이지 (RequestPage.tsx)

- 사용자가 메르카리(Mercari) 등 해외 쇼핑몰의 상품 URL을 입력하면 /api/products/fetch로 POST 요청을 보낸다.
- normalizeSoldOutFlags() 유틸 함수를 통해 동일한 productURL이 여러 번 들어온 경우 가장 앞의 1개만 “판매 가능”으로 유지하고 이후 동일 URL 상품은 강제로 isSoldOut = true 처리하여 중복 장바구니 추가를 방지한다.
- 로딩 상태에서는 스피너를 표시하고, 오류 발생 시에는 얼럿 등을 통해 사용자에게 명확히 안내한다.
- “장바구니에 담고 견적 확인하기” 버튼 클릭 시 선택된 상품들을 /api/cart에 추가하고 /cart로 이동한다.

마. 장바구니 및 견적 페이지 (CartPage.tsx + CartQuotation.tsx)

■ CartPage.tsx

- 마운트 시 /api/cart를 GET하여 현재 세션의 장바구니를 불러온다.

- 응답 데이터를 CartItem 형태로 매핑하여, 기본적으로 selected: true 상태로 초기화하여 전체 선택 상태로 표시한다.
- 전체 선택/해제, 개별 선택 토글, 선택 상품 삭제(/api/cart?ids=... DELETE), 개별 삭제 기능을 제공한다.
- “상품 추가하고 배송비 절약하기” 버튼 클릭 시 /request로 이동할 수 있다.
- 오른쪽 패널에서 추가 포장(extraPackaging), 보험(insurance) 옵션을 라디오 버튼 형태로 제공해 견적에 반영한다.

■ CartQuotation.tsx

- 실제 견적 계산을 담당하는 컴포넌트로, selectedItems(선택된 장바구니 상품 ID 리스트), extraPackaging, insurance를 props로 전달받는다.
- 이 값이 변경될 때마다 /api/cart/estimate에 POST 요청을 보내 견적을 다시 계산한다.
- 프론트엔드에서는 무게·부피 단위를 다음과 같이 가공해 표시한다.
 - totalActualWeightKg × 1000 → 예측 무게 (g)
 - totalVolumeM3 × 1,000,000 → 예측 부피 (cm³)
- 최종적으로 각 비용 구성 요소와 함께 [최종 결제 금액]을 카드형 UI로 보여주며, “총 결제 예상 금액 …원 결제하기” 버튼 클릭 시 /checkout으로 이동하는 콜백을 호출하도록 구현하였다.

바. 결제 페이지 (CheckoutPage.tsx)

■ 컴포넌트 마운트 시 다음 두 가지 요청을 순차적으로 수행한다.

- /api/cart GET: 실제 결제 대상 상품 목록을 가져와 orderItems로 저장한다.
- /api/cart/estimate POST: 동일 상품 목록에 대한 최종 견적을 다시 요청해 estimate로 저장한다.

■ 배송지 관리

- “배송지 등록” 버튼 클릭 시 AddressModal을 띄운다.
- AddressModal에서는 이름, 전화번호, 주소 검색 키워드, 상세주소, 요청사항을 입력받는다.
- /api/address/search로 키워드를 전달해 주소 목록을 받아오고, 선택된 항목을 도로명·우편번호 필드에 반영한다.
- validateAddress()로 각 필드를 검증한 뒤 문제가 없을 경우 /api/orders/address에 POST한다.
- 서버 응답으로 반환된 id, receiverName, phone은 추후 결제/주문 생성 단계에서 재사용할 수 있도록 localStorage에 저장한다.

■ 개인통관고유부호 관리

- “10초만에 조회하기” 버튼 클릭 시 CustomsCodeModal을 띄운다.
- validateCustomsCode()를 통해 P로 시작하는 13자리 형식인지 검증한다.
- /api/orders/customs-code/verify로 POST 요청을 보내 서버에서 유효성을 다시 확인한다.
- 검증이 통과되면 localStorage에 buylink_customsCode로 저장한다.

■ 결제 금액 표시 및 결제 실행

- estimate가 존재하는 경우 상품 금액, 대행 수수료, 해외·국내 배송비, 결제

수수료, 추가 포장/보험 비용을 구분해 표시하고 합계와 최종 결제 금액 (grandTotalKRW)을 강조한다.

- 예외적으로 estimate가 없는 경우 상품 금액 합계만을 기준으로 fallback 금액을 표시한다.
- 배송지, 개인통관고유부호, 약관 동의, 최종 금액이 모두 유효해야 결제를 진행한다.
- TossPayments SDK를 초기화한 뒤 requestPayment("CARD", {...})를 호출해 결제 창을 연다.
- 결제 성공 시 Toss가 /payments/success URL로 리다이렉션한다.

사. 결제 성공 처리 페이지 (PaymentsSuccessPage.tsx)

- Toss 리다이렉션으로 전달된 paymentKey, orderId, amount를 쿼리스트링에서 파싱해 사용한다.
- 1단계: 결제 검증 - /api/orders/pay로 { orderId, paymentKey, amount }를 POST하여 백엔드에서 실제 결제 금액과 키가 일치하는지 검증한다. 응답 상태 및 JSON을 상세 로깅해, 문제 발생 시 디버깅이 가능하도록 구성하였다.
- 2단계: 주문 생성 - Checkout 단계에서 localStorage에 저장해 둔 addressId, customsCode를 읽어온다. 결제 검증 결과의 totalAmount를 포함해 /api/orders에 POST하여 최종 주문을 생성한다.
- 3단계: 주문 완료 페이지 이동 - 서버에서 발급한 orderNumber(내부 주문번호)를 state로 넘겨 /order-complete로 이동한다. 오류 발생 시에는 화면에 메시지를 표시하고, “장바구니로 돌아가기” 버튼을 통해 /cart로 복귀할 수 있도록 구성하였다.

아. 주문 완료 페이지 (OrderCompletePage.tsx)

- URL 파라미터 또는 location.state로 전달된 orderId를 사용해 /api/orders/{orderId}를 조회한다.
- 화면 구성
 - 상단: “주문 완료” 안내 문구와 주문번호(복사 버튼 제공)
 - 좌측: 배송지 정보, 상품 목록, 결제 수단
 - 우측: 금액 상세(합계, 수수료, 최종 결제 금액)
 - 하단 버튼: “추가로 구매대행 요청”(/request), “홈으로 가기”(/)

자. 주문 내역 조회 페이지 (OrderHistoryPage.tsx)

- 사용자는 이름, 전화번호, 주문번호를 입력하여 과거 주문 정보를 조회한다.
- validateOrderHistory()를 통해 다음을 검증한다.
 - 이름: 공백 여부
 - 전화번호: 숫자 기준 10~11자리 여부
 - 주문번호: 숫자 기준 14자리 여부
- 검증 통과 시 /api/orders/{orderId}?receiver=...&phone=...으로 GET 요청을 보내 주문 내역을 조회한다.
- 응답 구조와 화면 구성은 주문 완료 페이지와 동일한 패턴을 재사용하여, 신규 주문 직후 화면과 과거 주문 조회 화면이 시각적으로 일관되도록 하였다.

차. 주요 개발 이슈와 해결 과정

■ 배포 및 백엔드 연동

- 이번 프로젝트에서 처음으로 실제 백엔드 서버와 연동하면서 배포에 대한 부분이 가장 막막했다. 이를 해결하기 위해 프론트엔드와 백엔드 모두 Vercel에 올려 GitHub 커밋 → 자동 배포가 이뤄지도록 구성했다. 그리고 API_BASE_URL을 정의해서 상대 경로(/api/...)를 사용해 Vercel의 리버스 프록시를 타도록 정리했다. 이렇게 공통 Base URL + 헬퍼 함수로 묶으면서 코드가 훨씬 단순해지고, GitHub에 커밋만 하면 자동으로 Vercel에서 프론트·백엔드가 같이 업데이트되도록 안정적인 흐름을 만들 수 있었다.

■ 구매대행 요청 페이지: 동일 상품의 품질 상태 처리

- 구매대행 요청 페이지에서는 같은 productURL이 여러 번 크롤링될 수 있어, 두 번째 요청부터는 무조건 품질로 처리하는 초기 접근 방식은 상품 삭제시 순서가 섞이거나 이미 품질인 상품이 중간에 섞여 들어오는 경우가 발생할 수 있어 단순 if문으로는 관리가 어려웠다. 그래서 같은 URL끼리 모아 첫 번째만 정상, 나머지는 isSoldOut = true로 만드는 유틸 함수로 분리해, 화면에서는 사용자가 요청을 여러 번 눌러도 첫 번째 상품만 장바구니에, 나머지는 품질 처리할 수 있었고, 품질 플래그 관련 로직도 페이지밖 유틸로 분리되면서 코드가독성이 좋아졌다.

■ 장바구니 페이지: 선택 상품만 견적 계산하기

- 삭제는 선택된 id만 모아 DELETE로 보내면 됐지만, 견적은 “전체 장바구니 기준”으로 짜여 있어 선택 상태와 itemIds를 맞추는 게 문제였다. 체크박스 상태를 selectedItems로 따로 관리하고, 견적 API 호출 시에만 selectedItems로 itemIds를 구성하도록 바꿔 삭제/견적이 동일한 선택 상태를 공유하고, 실제 견적도 사용자가 체크한 상품 기준으로 계산되도록 했다.

■ 유효성 검증

- 잘못된 입력으로 결제·주문 API가 호출되지 않도록 공통 유효성 검증 모듈을 두고, 폼 제출 시 errors 객체를 만든 뒤 가장 첫 에러만 사용자에게 보여주도록 했다. 이렇게 프론트에서 1차 필터링을 수행해 형식이 맞지 않는 값은 아예 API 요청 단계까지 가지 못하게 막았다.

■ 결제 이후 페이지 전환: TossPayments 연동과 결제 검증 흐름

- /payments/success 진입 후 “결제 결과 처리 중입니다”에서 멈추는 문제를 해결하기 위해, 결제 이후 흐름을 ① /api/orders/pay로 결제 검증(paymentKey, orderId, amount 검증) ② /api/orders로 주문 생성(addressId, customsCode, totalAmount 포함) ③ orderNumber를 들고 /order-complete로 이동하는 세 단계로 분리했다. 이 구조로 재구성한 뒤 “결제 승인 → 검증 → 주문 생성 → 주문 완료 화면”까지 끊김 없이 이어지며, 사용자가 결제 직후 바로 자신의 주문 상세를 확인할 수 있게 되었다.

카. UI·UX 설계 및 향후 개선 방향

- 프론트엔드 구현뿐만 아니라 전체 UI·UX 설계를 담당하여, Figma 시안(PM이 제공한 앱 디자인)을 바탕으로 컬러·버튼·카드 레이아웃을 정리하고 TailwindCSS로 메인-요청-장바구니-결제-주문 완료/조회 전 구간의 디자인

언어를 통일했으며, 결제 금액 박스·상품 카드·주소 영역을 공통 패턴으로 묶고 반응형 그리드·여백 설계로 데스크톱·노트북 환경에서 깨지지 않는 레이아웃을 구현했다. 필수 입력 누락 시 버튼 비활성화·안내 문구, 로딩/빈/에러 상태별 전용 화면으로 상태를 명확히 보여주도록 했고, 앞으로는 PaymentSummaryCard, OrderItemList, AddressBlock 같은 공통 컴포넌트와 디자인 토큰(색상·폰트·간격) 정리, 렌더링 최적화로 옵션 변경 시 발생하는 미세한 깜빡임·버벅임을 줄이는 방향으로 개선할 계획이다.

8. AI

가. 프로젝트 개요 및 목표

- Stage 1-2에서 확정적 정보를 우선 확보하고, 실패 시 Stage 3의 AI 모델 사용
- 3-Stage 파이프라인 구조

단계	처리 내용	출력	신뢰도 기준
Stage 1	OCR 텍스트 추출	무게/부피 텍스트 패턴	이중 OCR 교차검증 85%+
Stage 2	제품 스펙 매칭	공식 제품 스펙	텍스트-이미지 일치 85%+
Stage 3	AI 무게/부피 예측	카테고리/무게/부피	모델 신뢰도 출력

나. 데이터 수집

- AIHub 물류공간 예측 데이터셋
 - 총 데이터 수: 122,378건
 - 데이터 분할: 학습(8) : 검증(1) : 테스트(1)
 - 포함 정보: 제품 이미지, 바운딩 박스, 무게(kg), 부피(L/W/H), 카테고리
- 메루카리(Mercari) 크롤링 데이터셋
 - 일본 최대 중고거래 플랫폼 메루카리에서 실제 판매 상품 데이터를 직접 수집
 - 메루카리 파이썬 라이브러리를 활용하여 크롤링을 진행
 - 수집 조건:
 - 상품 이미지 3개 이상 보유(다각도 촬영)
 - 상품 설명글에 무게 또는 부피 정보 포함
 - 각 소분류 카테고리별 최대 50개 수집
 - 수집 결과:
 - 총 수집량: 9,914개 제품
 - 대분류: 22개(예: 게임/장난감/굿즈, 책/잡지/만화, 가전/스마트폰 등)
 - 중분류: 82개(예: 피규어, 만화, 생활가전 등)
 - 소분류: 323개(예: 애니메이션 피규어, 소년만화, 주방가전 등)
 - 데이터 구조: 제품ID, 이름, 가격, 무게(g), 치수(mm), 부피(cm³), 카테고리 계층, 이미지 파일명
- 카테고리별 통계 분석
 - 수집된 메루카리 데이터를 카테고리별로 분류하여 무게/부피 통계 산출
 - 이 통계는 AI 모델의 예측 정확도를 높이는 핵심 참조 데이터로 활용

카테고리	샘플 수	평균 무게(g)	표준편차(g)	최소(g)	최대(g)
피규어	487	245.3	182.4	15	1,850
만화책	312	198.7	45.2	120	380
스마트폰	156	189.2	28.6	142	285
생활가전	234	1,247.8	892.3	180	5,200
의류	521	312.4	156.8	45	1,200

- 통계 활용 방법:

- MAST3R 스케일 보정: 3D 복원 결과의 상대적 크기를 카테고리 평균 부피로 정규화
- GPT 프롬프트 참조: '이 카테고리의 평균 무게는 Xg입니다'와 같이 참조값 제공
- 이상치 탐지: 예측값이 카테고리 범위(평균 $\pm 2\sigma$)를 벗어나면 재검토

다. Stage 1-2 시스템 개발

■ Stage 1: 텍스트 정보 추출

- 판매자가 작성한 설명란에서 무게/부피 정보를 직접 추출하는 규칙 기반 시스템
- 정규표현식 기반 다국어(일/영/한) 무게/부피 패턴 정의
- Google Vision API와 EasyOCR 이중 OCR 양상블
- 교차 검증하여 신뢰도 85% 이상인 경우 결과 채택

■ Stage 2: 제품 스펙 매칭

- Stage 1이 실패한 경우, 제품명과 사진을 이용해 공식 제품 스펙을 검색하는 시스템
- NER 모델로 제품명 추출웹검색공식 스펙 확인
- CLIP 모델로 텍스트-이미지 일치도 검증(85% 이상)
- FAISS 벡터DB로 유사 제품 검색(fallback)

라. Stage 3 Image2Mass 모델

■ 모델 설계

- 3D 복원 기반 부피 추정 후 무게를 예측하는 방식
- MAST3R 3D 복원파이프라인

단계	처리 내용	상세 설명
1	이미지 로드 및 전처리	다각도 이미지를 512x512 해상도로 리사이즈, RGB 정규화
2	이미지 페어 생성	Complete graph 방식으로 $N*(N-1)/2$ 개의 페어 생성
3	ViT 인코더 추론	각 페어에 대해 ViT-Large 백본으로 깊이맵 및 특징점 추출
4	Global Alignment	Bundle Adjustment로 카메라 포즈 및 깊이맵 최적화
5	포인트 클라우드 생성	신뢰도 기반 필터링 적용, 3D 좌표 추출
6	바운딩 박스 계산	IQR 기반 이상치 제거 후 축 정렬 바운딩 박스로 L, W, H 추출

- Image2Mass 파이프라인

모듈	구성	출력
Backbone	Xception (ImageNet 사전학습)	2048-dim 이미지 특징 벡터
Geometry Module	두께 마스크(19x19 그리드) + FC layers	14-dim 기하학적 특징
Density Tower	이미지 특징 → FC(512) → FC(256) → FC(1)	밀도 백분위수 (0~1)
Volume Tower	기하학적 특징 → FC(64) → FC(32) → FC(1)	부피 예측값 (cm³)
최종 출력	Mass = Density × Volume	무게 (g)

- 핵심 문제 발견: MAST3R는 참조 물체 없이 절대 미터 스케일 추정 불가. 각 뷰별 깊이맵의 스케일 범위가 0.4~1.1m로 상이.

■ 모델 학습

- MAST3R 스케일 보정을 위해 메루카리 카테고리별 평균 부피값을 참조
- 버전1 - 기본: MAST3R로 3D 복원 후 바운딩 박스에서 부피를 직접 추출.
- 버전2 - 카테고리 예측 추가: 앞단에 카테고리 예측 모델을 배치하여 해당 카테고리 평균값 참조.
- 버전3 - 카테고리 정답값 사용: 카테고리 예측 모델 생략, 정답 카테고리값 직접 입력.

■ 모델 평가

- 메루카리 1,000 샘플을 대상으로 3가지 버전의 성능을 평가하였다.
- [표3] MAST3R + Image2Mass 버전별 성능 비교

버전	전략	성공률	MAE (g)	500g+	비고
버전1	기본(MAST3R + Image2Mass)	100%	1,247.3	42.0%	스케일 추정 실패
버전2	카테고리 예측+ MAST3R	100%	834.3	39.0%	예측 오류 전파
버전3	카테고리 정답값+ MAST3R	100%	623.5	30.0%	최선

마. Stage 3 GPT 모델

■ 모델 설계

- GPT-4o의 비전 기능과 사전 학습된 제품 지식을 활용
- GPT-4o Vision API (detail=high, temperature=0.3)
- 메루카리 카테고리별 평균/표준편차 통계 제공
- 다양한 프롬프트 전략 실험

■ 모델 학습

- GPT-4o는 사전 학습 모델이므로 프롬프트 최적화를 통해 성능 개선
- Baseline: 기본 프롬프트로 무게/부피 예측 요청.
- v2 - 상세 카테고리 통계: 세분화된 카테고리별 평균/표준편차 제공.
- v3 - 카테고리 참조: 카테고리 평균값을 '참조'로만 제공하고 GPT가 이미지 기반 조정.
- Few-Shot: 카테고리별 3개 예시 제공.
- v4 - 사전지식 우선: '제품을 정확히 알면 실제 스펙으로, 모르면 카테고리 평균 사용' 하이브리드 전략.
- v5 - Web Search: 웹 검색 API로 실시간 제품 정보 검색.

■ 모델 평가

- 메루카리 1,000 샘플을 대상으로 6가지 버전의 성능을 평가하였다.

버전	전략	성공률	MAE (g)	500g+	비고
Baseline	기본 프롬프트	94%	892.4	35.0%	-
v2	상세 카테고리 통계	98%	1,124.7	45.2%	과도한 세분화
v3	카테고리 참조	100%	412.6	20.0%	안정적
Few-Shot	카테고리별3예시	98%	567.3	26.5%	가전 오류
v4	사전지식 우선	100%	165.3	8.0%	최고
v5	Web Search	32%	-	-	실패

- v2: 과도하게 세분화된 카테고리가 매칭 오류 유발
- Few-Shot: 잘못된 예시가 극단적 오류 유발
- v5:JSON 파싱 오류32%, 제품 오인식
- v4(사전지식 우선) 상세 분석
 - 하이브리드 전략: '제품을 정확히 알면 실제 스펙으로 답하고, 모르면 카테고리 평균을 폴백으로 사용'
 - High Confidence (28%): 정확한 제품 인식실제 스펙 응답
 - Medium Confidence (68%): 유사 제품 추론카테고리 기반 추정
 - Low Confidence (4%): 불확실카테고리 평균 폴백

바. 두 모델 성능 비교

■ Image2Mass 기반 방식과GPT-4o Vision 방식의 성능을 종합 비교

방법	성공률	MAE (g)	500g+ 오차	순위
Image2Mass + MAST3R (기본)	100%	1,247.3	42.0%	8
Image2Mass + 카테고리 예측	100%	834.3	39.0%	7
Image2Mass + 카테고리 정답값	100%	623.5	30.0%	5
GPT Baseline	94%	892.4	35.0%	6
GPT v3 (카테고리 참조)	100%	412.6	20.0%	2
GPT v4 (사전지식 우선)	100%	165.3	8.0%	1
GPT Few-Shot	98%	567.3	26.5%	3
GPT v5 (Web Search)	32%	-	-	9

■ 고찰

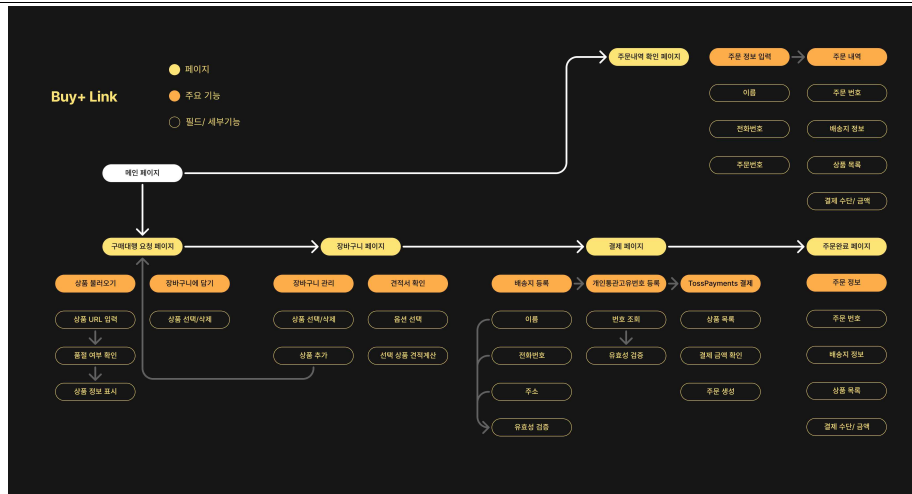
- GPT 사전지식 효과: '알면 답하라'고 명시적으로 지시하면 성능이 극적으로 향상
- 카테고리 정보 활용: 참조용으로 제공 시 효과적, 과도한 세분화는 역효과
- 3D 복원 한계: MAST3R는 참조 물체 없이 절대 스케일 복원 힘들

■ 최종 모델 선정

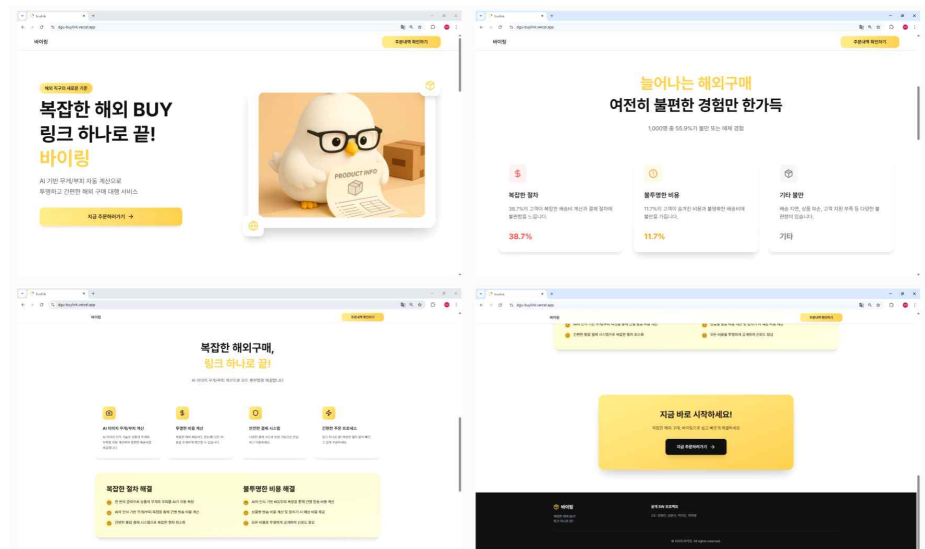
- 프로덕션 배포 모델: GPT v4 (사전지식 우선)
- 500g+ 오차율8.0% - 배송비 산정에 가장 실용적
- MAE 165.3g - 가장 작은 절대 오차

3.3 결과분석

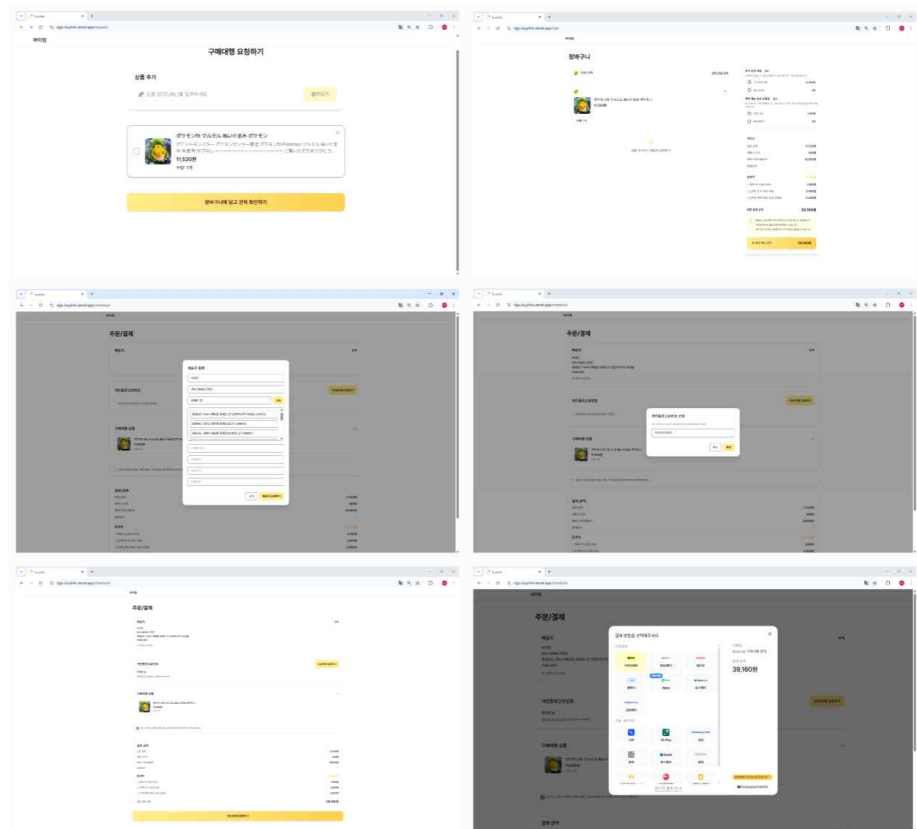
가. 사용자 흐름



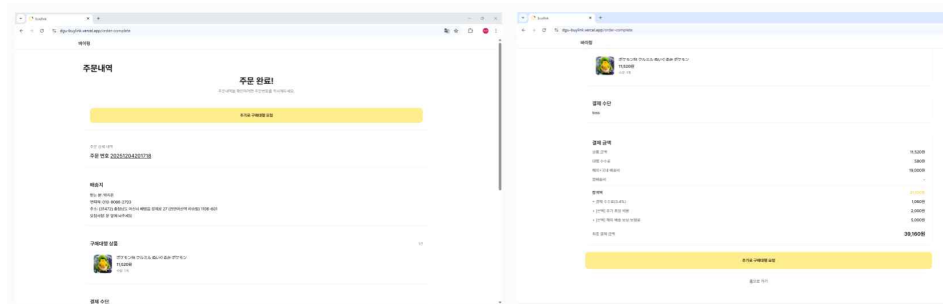
- 사용자는 메인 페이지에서 서비스 소개와 핵심 가치를 확인한 뒤, “지금 주문하러 가기” 버튼을 통해 구매대행 요청 흐름을 시작한다.



- 구매대행 요청 페이지에서 해외 쇼핑몰(메루카리 등)의 상품 링크(URL)를 입력해 상품 정보를 불러오고, 장바구니에 담아 견적을 확인한다.
- 장바구니 페이지에서는 여러 상품을 한 번에 관리하면서 옵션 선택에 따른 예상 비용 변화를 비교해 본 뒤, 결제 페이지로 이동해 실제 결제를 진행한다.



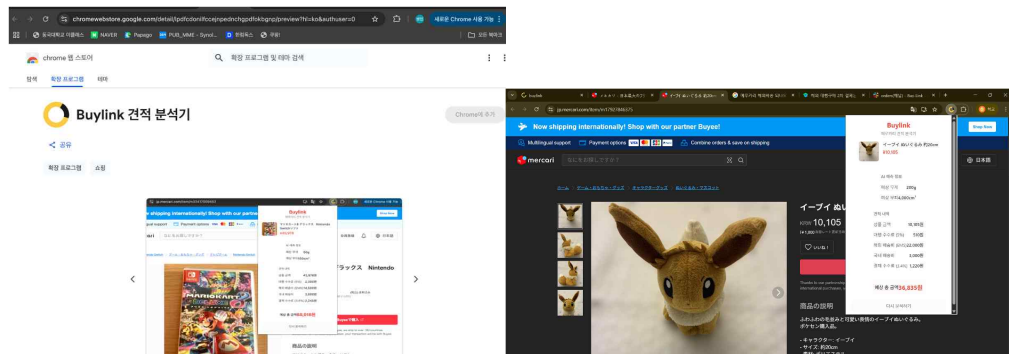
- 결제가 완료되면 주문 완료 페이지에서 전체 주문 내역을 다시 확인할 수 있고, 이후에는 주문 내역 조회 페이지에서 이름 · 전화번호 · 주문번호만으로 과거 주문을 다시 찾아볼 수 있도록 사용자 흐름을 설계하였다.



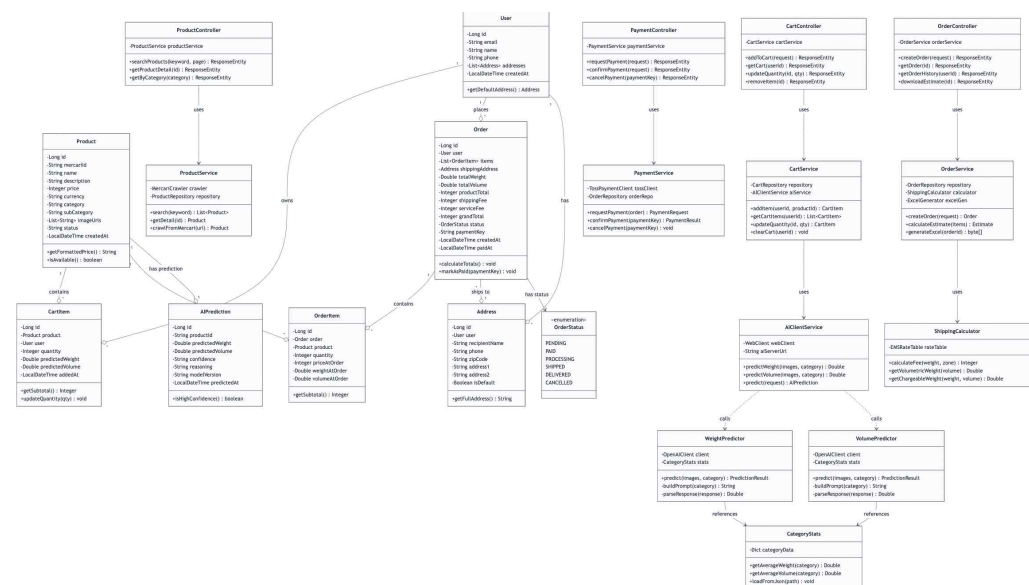
- 구매자가 결제가 완료하면 관리자가 사용하는 Slack(채팅앱)에 알림이 가고 결제 정보와 견적서가 보인다. 또한 서비스 운용중에 오류가 발생해도 슬랙 챗봇이 이를 알린다.



■ 추가로 사용자는 google chrome 확장프로그램으로 Bulink 견적 분석기를 다운받으면 메루카리 페이지 안에서도 AI 견적 분석이 가능하다



나. 클래스 다이어그램



다. 실제 고객 테스트

- 10명의 실제 고객을 대상으로 바이링 서비스 테스트를 진행하고. 실제 수익을 얻음

테스트	고객	상품	예측 무게	실제 범위	결과
1	민*진	산리오 피규어 + 키링	420g	<500g	성공
2	박*령	중고 아이폰	172g	<500g	성공
3	김*연	도라에몽 인형 21개 + 피규어	1,200g	1-1.5kg	성공

라. 서비스 구성

서비스	기술 스택	포트	역할
Frontend	React 19, TypeScript, Vite	Vercel	사용자 인터페이스
Backend	Spring Boot 3.5, Java 17	17788	API 서버, 비즈니스 로직
AI	Python 3.11, Flask, OpenAI	7001	무게/부피 예측
Crawler	Python 3.11, Selenium, Chrome	5001	메르카리 크롤링
Database	MariaDB	3306	데이터 저장

사용자 요청은 Frontend(Vercel)를 통해 Backend로 전달되며, Backend는 필요에 따라 Crawler(상품 정보 수집), AI(무게/부피 예측), MariaDB(데이터 저장)와 통신한다. 결제 완료 시 Slack API를 통해 관리자에게 알림과 견적서가 자동 발송된다.

마. 코드 구조화 및 모듈화

3.1 Backend (Spring Boot) - 계층형 아키텍처

```
BE/src/main/java/io/github/hayo02/proxyshopping/
├── common/           # 공통 유틸리티 (ApiResponse.java)
├── cart/             # 장바구니 도메인
│   ├── controller/  # REST Controller
│   ├── service/     # 서비스 인터페이스
│   └── serviceImpl/ # 서비스 구현체
├── repository/       # JPA Repository
├── entity/           # JPA Entity
├── dto/              # Data Transfer Objects
├── orders/           # 주문 도메인 (동일 구조)
└── exception/        # 전역 예외 처리
```

설계 원칙

- 인터페이스-구현체 분리: 서비스 계층을 interface 와 impl로 분리하여 의존성 역전
- 도메인 기반 패키지 구조: cart, orders 등 도메인별 패키지 구성
- DTO 패턴: Entity 와 API 응답을 분리하여 캡슐화

3.2 Frontend (React) - 기능별 구조

```
FE/buylink/src/
├── pages/           # 페이지 컴포넌트
│   ├── MainPage.tsx
│   ├── RequestPage.tsx
│   ├── CartPage.tsx
│   ├── CheckoutPage.tsx
│   └── StatusPage.tsx # 모니터링 대시보드
├── components/      # 재사용 컴포넌트
├── hooks/           # Custom Hooks
└── layouts/         # 레이아웃 컴포넌트
```

3.3 AI/Crawler - 단일 책임 구조

```
AI/
├── api_server_standalone.py ── Crawler
├── category_stats.json       ── app.py
├── requirements.txt          ── mercari_crawler_2.py
└── Dockerfile               ── requirements.txt
```

바. 사용된 오픈소스 라이브러리

- BE

라이브러리	버전	용도	라이선스
Spring Boot	3.5.6	웹 프레임워크	Apache 2.0
Spring <u>WebFlux</u>	-	비동기 HTTP 클라이언트 (Slack)	Apache 2.0
Spring Data JPA	-	ORM, 데이터베이스 접근	Apache 2.0
Lombok	-	<u>보일러플레이트 코드</u> 감소	MIT
Apache POI	5.2.5	Excel 견적서 생성	Apache 2.0
Jackson	2.17.1	JSON 처리	Apache 2.0
MariaDB Connector	-	데이터베이스 드라이버	GPL 2.1

- FE

라이브러리	버전	용도	라이선스
React	19.1.1	UI 프레임워크	MIT
TypeScript	5.9.3	정적 타입	Apache 2.0
Vite	7.1.7	빌드 도구	MIT
React Router	7.9.5	클라이언트 라우팅	MIT
Recoil	0.7.7	상태 관리	MIT
Axios	1.13.1	HTTP 클라이언트	MIT
<u>TailwindCSS</u>	4.1.16	CSS 프레임워크	MIT
Motion (Framer)	12.23.24	애니메이션	MIT

- AI

라이브러리	버전	용도	라이선스
Flask	3.0+	웹 프레임워크	BSD-3
OpenAI	1.0+	GPT-4o API 클라이언트	MIT
Selenium	4.15+	웹 브라우저 자동화	Apache 2.0
WebDriver Manager	4.0+	<u>ChromeDriver</u> 자동 관리	Apache 2.0
<u>Gunicorn</u>	21.0+	WSGI 서버	MIT

4. 기대효과

9. 기술적 측면

가. AI 기반 무게/부피 예측 기술 고도화

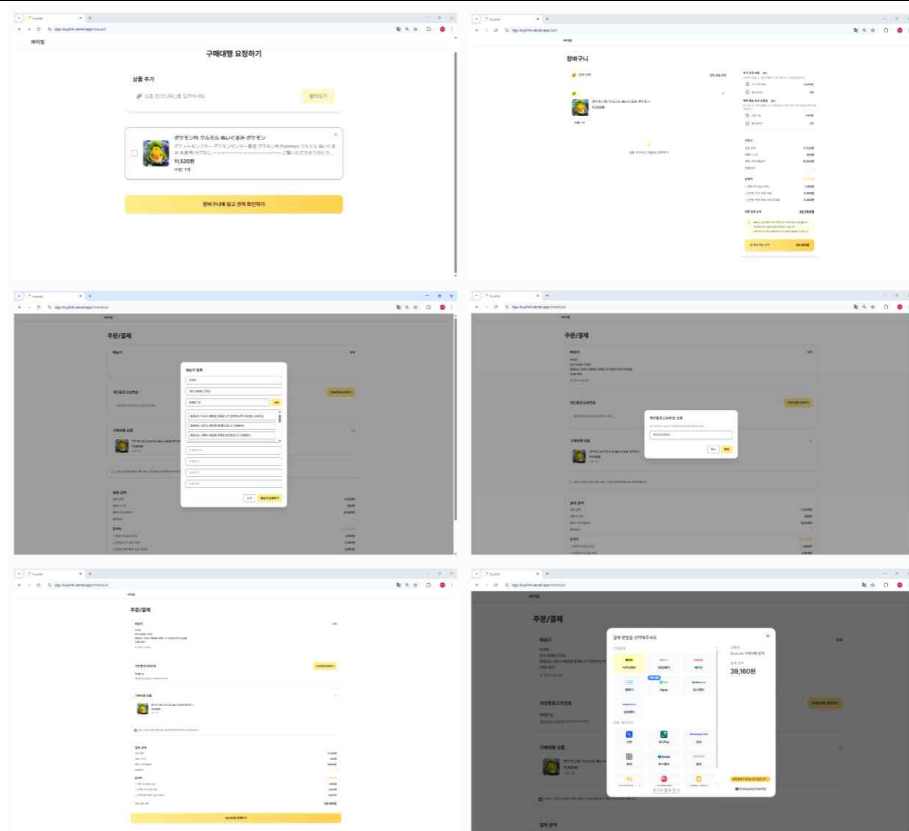
- 기존 무게, 부피 측정 후 가격을 산정하는 인력 의존도를 90% 이상 감소
- 예측 정확도를 높여 인력 의존도를 0으로 만드는 것이 최종 목표

나. 통합 결제, 배송 및 통관 자동화 기술 실현

- 상품 링크 입력만으로 결제와 배송이 자동화 되는 구매대행 구조를 통해 기존 절차를 50% 감소시키며, 평균 10분이 걸리던 절차가 10초로 단축됨
- 사람이 견적을 내고, 사람이 확인하여 배송하는 구조에서 결제 오류율을 30% 이상 단축시킬 수 있음
- 정밀한 배송비 예측 모델 고도화로 고객 CS 처리시간이 70% 이상 단축되며 오차로 인해 손해보는 물류비가 15% 이상 감소함

	다. 경제 산업적 측면 ■ 구매대행 산업의 디지털 전환 촉진 - 기존 수동 및 수기 중심의 구매대행 업무를 AI 자동화로 전환 - 구매대행 업체의 비용과 인력 절감을 통해 운영비를 30% 절감, 처리속도는 2배 이상 향상 ■ B2B 확정으로 인한 산업 파급효과 - 타 구매대행사에 모델 이용료를 받으며 매출 20% 이상 확보 가능 - 역직구로 확장하여 인디 캐릭터 및 로컬 굿즈 브랜드와의 협업 가능 10. 활용 방안 가. 커머스 플랫폼 분야 ■ 자체 자사몰 서비스 상용화 후 도소매와 함께 구매대행 AI 모델 API 연동 ■ 도소매와 구매대행 병행을 통해 고객 구매 데이터를 확보하여 추천 알고리즘을 개발하거나, 소비자 취향 타겟 상품을 직접 제조해 매출 증대 나. 글로벌 확장 ■ 일본, 싱가포르, 미국 등 한국 직구를 희망하는 사용자에게 한국 상품 구매대행 서비스 제공 가능 (역직구)
5. 산학협력	* 멘토의 역할 및 멘토링 지도 내역을 구체적으로 기술 요망★ (멘토링, 멘토 활용내역)
6. 참고문헌	[1] T. Standley, O. Sener, D. Chen, and S. Savarese, "<i>image2mass: Estimating the mass of an object from its image</i>," in <i>Proc. Conf. Robot Learning (CoRL)</i>, 2017, pp. 324-333. [Online]. Available: https://proceedings.mlr.press/v78/standley17a.html [2] A. Kirillov, E. Mintun, N. Ravi, H. Mao, C. Rolland, L. Gustafson, et al., "<i>Segment Anything</i>," in <i>Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)</i>, 2023, pp. 4015-4026. [Online]. Available: https://openaccess.thecvf.com/content/ICCV2023/papers/Kirillov_Segment_Any

	thing_ICCV_2023_paper.pdf [3] C. Wimalasiri and P. K. Sahoo, "Vision-Based Approach for Food Weight Estimation from 2D Images," <i>arXiv preprint arXiv:2405.16478</i>, 2024. [Online]. Available: https://arxiv.org/abs/2405.16478 [4] D. J. Díaz-Romero, H. J. Escalante, and R. A. Gómez-González, "Simultaneous mass estimation and class classification of scrap metals using deep learning," <i>Resources, Conservation and Recycling</i>, vol. 181, p. 106272, 2022. doi: 10.1016/j.resconrec.2022.106272 [5] G. Vinod, J. He, Z. Shao, and F. Zhu, "Food Portion Estimation via 3D Object Scaling," <i>arXiv preprint arXiv:2404.12257</i>, 2024. [Online]. Available: https://arxiv.org/abs/2404.12257 [6] R. Ranftl, K. Lasinger, D. Hafner, K. Schindler, and V. Koltun, "Towards Robust Monocular Depth Estimation: Mixing Datasets for Zero-shot Cross-dataset Transfer," <i>IEEE Trans. Pattern Anal. Mach. Intell.</i>, vol. 44, no. 10, pp. 6383-6398, 2020. doi: 10.1109/TPAMI.2020.3019967 [7] Z. Shao, G. Vinod, J. He, and F. Zhu, "An End-to-end Food Portion Estimation Framework Based on Shape Reconstruction from Monocular Image," <i>arXiv preprint arXiv:2308.01810</i>, 2023. [Online]. Available: https://arxiv.org/abs/2308.01810 [8] AI Hub. (2024). 물류공간 예측 데이터셋. https://www.aihub.or.kr/ [9] Mercari, Inc. Mercari Japan Marketplace. https://jp.mercari.com/ [10] 한국소비자원. (2024). 해외직구 관련 소비자 불만 현황 분석. 소비자24. https://www.consumer.go.kr/user/ftc/consumer/cnsmrBBS/740/selectInfoCDMGCSLCDetail.do?infoId=A1080210&cntntsId=00000347 [11] 한국소비자원. (2019). 해외직구 소비자 불만 동향 및 피해예방 가이드라인. 스마트컨슈머 리포트. https://www.kca.go.kr/smartconsumer/sub.do?menukey=7301&mode=view&no=1002768687&page=13
7. 첨부	1. 팀원 및 멘토가 함께 찍은 사진 1장 jpg  2. 작품설명용 사진 1~2장 jpg (ex. 4.개념설계.jpg) 4장 참조



3. 시연동영상 링크

<https://www.youtube.com/watch?v=azo4WCGvIL0>

[양식1-1] 오픈소스경진대회 프로젝트 최종보고서

프로젝트 결과보고서 [요약]

과 제 명	바이링_해외구매 간소화 서비스		
협력기관명		과제책임자	송수환 교수
참여인원	(총 6명) 기업체 1명, 참여교수 1명, 학부과정 4명		
	구분	성명	정보
	산업체	최상철	소속: 솔루션
			직함: 상무
	교수	송수환	컴퓨터공학부 교수
	학생	강병민	멀티미디어소프트웨어공학부 학부생
		남윤수	컴퓨터공학전공 학부생
		박지은	컴퓨터공학전공 학부생
		최하영	컴퓨터공학전공 학부생
수행기간		소요비용	
추진배경	국내 해외직구 시장은 지속적으로 성장하고 있으나, 구매 과정이 복잡하고 배송비·수수료 등 최종 비용을 사전에 알기 어려워 소비자 불편과 피해가 반복되고 있다. 특히 기존 구매대행 서비스는 단단계 절차와 2차 결제 구조로 인해 시간과 비용 부담이 크다. 본 프로젝트는 이러한 구조적 문제를 기술적으로 개선하기 위해 추진되었다.		
목표 및 내용	본 프로젝트의 목표는 AI 기반 자동화 기술을 활용해 해외 구매대행 절차를 단순화하고, 배송비와 세금을 포함한 총비용을 결제 전에 예측할 수 있는 구매대행 플랫폼을 구축하는 것이다. 상품 링크 또는 이미지 입력만으로 상품 정보 수집, 무게·부피 예측, 자동 견적 계산, 원클릭 결제가 가능하도록 시스템을 설계·구현하였다. 이를 통해 사용자 개입과 수작업 중심의 기존 절차를 최소화하고자 하였다.		
수행결과	본 프로젝트를 통해 AI 기반 해외 구매대행 자동화 서비스 바이링(Buylink)을 구현하였다. 상품 정보 크롤링, AI 기반 무게·부피 예측, 자동 견적 계산, 장바구니 관리, 결제 및 주문 내역조회 기능을 하나의 서비스 흐름으로 통합하였다. AI 시스템은 상품 이미지와 카테고리 정보를 기반으로 무게·부피를 예측하고, 이를 배송비 산정 로직과 연동하여 결제 전에 총비용을 산출하도록 설계되었다. 메루카리 상품 1,000건을 대상으로 테스트한 결과, 무게 예측 오차 500g 이내 기준에서 실사용 가능한 예측 정확도를 확인하였으며, 기존 구매대행 서비스의 2차 결제 구조를 기술적으로 대체할 수 있음을 검증하였다. 또한 자동 견적 계산을 통해 기존 구매대행에서 수작업 견적 절차를 단축하였다. 로그인 없이 세션 기반으로 구매 흐름이 유지되도록 구현하였고, 결제 단계에서는 서버 검증을 통해 화면 표시 금액과 실제 결제 금액의 일치 여부를 확인함으로써 결제 오류 가능성을 줄였다. 종합적으로 본 프로젝트는 AI 기반 무게·부피 예측 기술을 구매대행 구조에 적용해 비용 예측의 불확실성과 절차적 복잡성을 동시에 개선하였으며, 향후 물류·커머스 분야로 확장 가능한 기술적 가능성을 확인하였다.		